



MMAUG / AI & DEVOPS BOOTCAMP

Learn the ABCD of Git & GitHub for AI Apps

A visual beginner handbook + terminal-first collaboration

Git, GitHub, review and reproducible AI application practice

Participant labs: use your own practice repository

A Global Community for AI, Cloud, Infra, DevOps and Tech Enthusiasts

Learn. Innovate. Build.

MMAUG community



mmaug.com



Agenda: the ABCD learning journey

Concepts on screen; detailed exercises in the public participant workbook.

- | | | | |
|----|------|---------------------------|--|
| 01 | ———— | A — Acquire | What Git is, installation, identity, staging and commits |
| 02 | ———— | B — Branch | Branching strategies, merge/rebase, conflicts and safe recovery |
| 03 | ———— | C — Collaborate | GitHub remotes, Issues, Projects, PRs and branch rulesets |
| 04 | ———— | D — Deliver | Introductory Actions, configuration, OIDC and failed-run diagnosis |
| 05 | ———— | Practice + Q&A | Labs 1–5, optional governance practice and participant resources |



Git & GitHub: what are they?

Git records the history. GitHub hosts that history and the collaboration.

1. What is Git?

A distributed version control system (DVCS).
Records snapshots called commits; supports offline work.
Tracks who changed what, when and why.

2. Why use version control?

Track changes and recover earlier versions.
Collaborate without casually overwriting others' work.
Experiment safely; keep a reviewable history.

3. What is GitHub?

A cloud platform that hosts Git repositories.
Adds PRs, reviews, Issues, Projects and Actions.
Git works with or without GitHub.

VCS type	Example	How it stores history
Local	RCS	One local history
Centralized	SVN	A central server
Distributed	Git; Mercurial	Local histories + remotes

Analogy: each commit is a named stop on your project's timeline.



Git hosting: compare the platforms

Same Git fundamentals; different collaboration tools, pipelines and ecosystems.

GitHub

Review: pull requests

Plan: Issues + Projects

CI/CD: GitHub Actions

Typical fit: open source and broad collaboration

Azure Repos

Review: pull requests

Plan: Azure Boards (same DevOps suite)

CI/CD: Azure Pipelines (same DevOps suite)

Typical fit: teams using Azure DevOps

GitLab

Review: merge requests

Plan: issues + issue boards

CI/CD: GitLab CI/CD

Typical fit: an integrated DevSecOps platform

Bitbucket

Review: pull requests

Plan: Jira integration

CI/CD: Bitbucket Pipelines (Cloud)

Typical fit: teams using Jira and Atlassian

Git runs locally. These platforms host repositories and support collaboration.

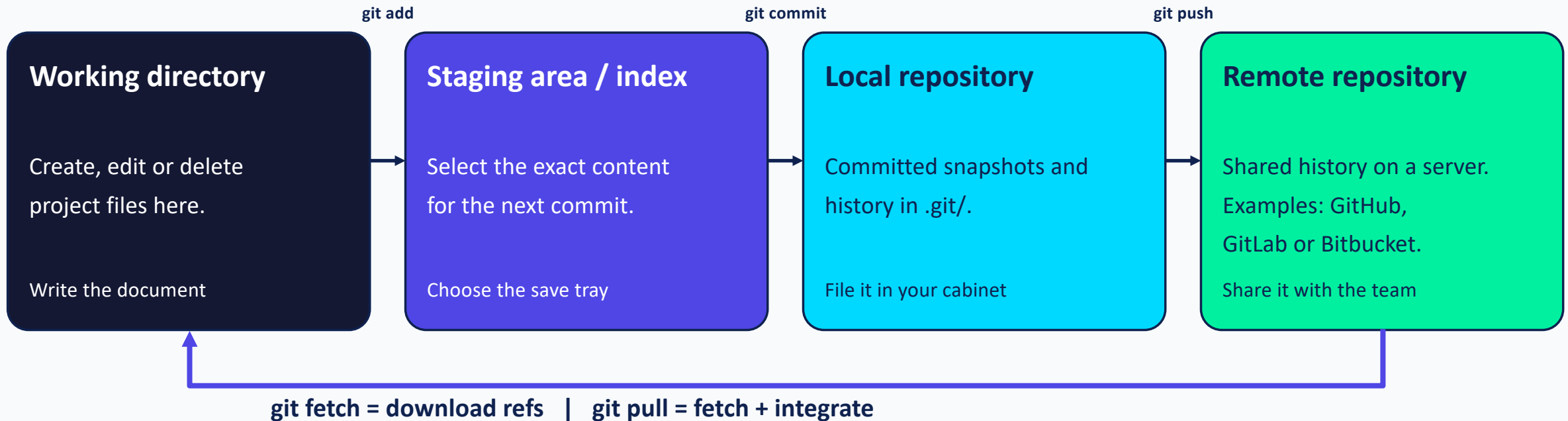
Choose for team needs and integrations; GitHub is our workshop platform. Features vary by edition and configuration.





Git architecture: four places your work can live

Edit → stage → commit → push. Fetch downloads; pull also integrates.



```
.git/  
HEAD  objects/  refs/  config  
index logs/    hooks/
```

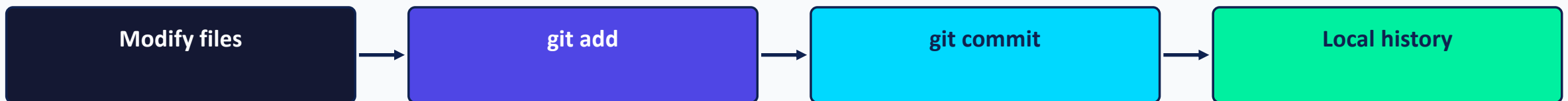
Do not edit `.git` by hand.
Commit locally first; push to share.



Essential Git commands: create, stage, commit

Every command has a purpose. The staging area lets you choose what belongs together.

Command	Purpose / what changes	Beginner checkpoint
git init -b main	Start a local repository with main	A hidden .git directory appears
git clone REPO_URL	Copy an existing repository	origin is usually set automatically
git status	Show branch + tracked/untracked file states	Read before and after each action
git add README.md	Stage one file for the next snapshot	Use git add -p to select hunks
git commit -m "Add prompt template"	Record the staged snapshot locally	A new commit ID is created





Inspect history and configure your commit identity

Important: `git config user.name / user.email` sets author metadata, not authentication.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --list --show-origin
```

Command	Use it to answer
<code>git diff / git diff --staged</code>	What changed before staging / in the next commit?
<code>git log --oneline --graph --all</code>	How did commits and branches develop?
<code>git show COMMIT</code>	What does a particular commit contain?
<code>git show --name-only COMMIT</code>	Which paths changed in this commit?
<code>git help COMMAND</code>	What options and examples does this command support?



Install Git + gh; authenticate in VS Code

Git = local version control. gh = GitHub CLI. Install both, then reopen the terminal.

Platform	Install Git + GitHub CLI
Windows	winget install --id Git.Git -e winget install --id GitHub.cli -e
macOS + Homebrew	brew install git gh
Linux (Debian/Ubuntu)	sudo apt update && sudo apt install git For gh: cli.github.com → Linux
Verify the installation	git --version / gh --version

```
gh auth login --hostname github.com \  
  --git-protocol https --web  
gh auth status  
gh auth setup-git
```

Two separate identities

git config = who authored a commit.

gh auth = permission to use GitHub.

SSH / HTTPS helper = Git transport access.

Do not paste passwords, PATs or private SSH keys into slides, source code or command history.



Remote repositories: connect and synchronize

A remote is another repository. origin is a conventional name, not a special Git feature.



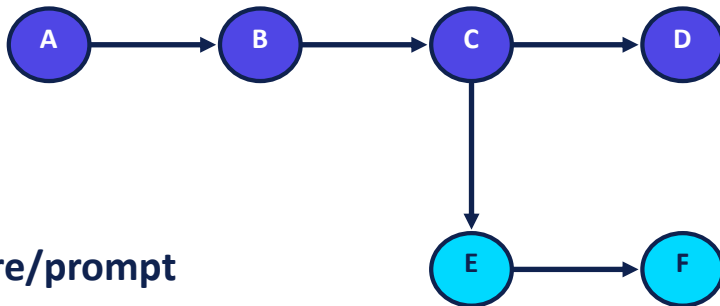
Connect / inspect	Synchronize / publish
<code>git remote add origin REPO_URL</code>	<code>git push -u origin main</code>
<code>git remote -v</code>	<code>git fetch origin</code>
<code>git remote set-url origin REPO_URL</code>	<code>git pull --ff-only</code>
<code>git remote rename origin team</code>	<code>git push -u origin feature/prompt</code>

clone = copy + set origin | fetch = download only | pull = fetch + integrate | push = publish

Git branching: a pointer, not a folder copy

A branch is a lightweight, movable reference to a commit. Work safely without changing main.

main



feature/prompt

A–C are shared history; D and E/F diverge.

```
git branch
git branch -a
git switch -c feature/prompt
git branch --show-current
git switch main
git branch -m old-name new-name
git branch -d feature/prompt
```

Update main

Create branch

Edit + commit

Push + PR

Merging: bring histories together

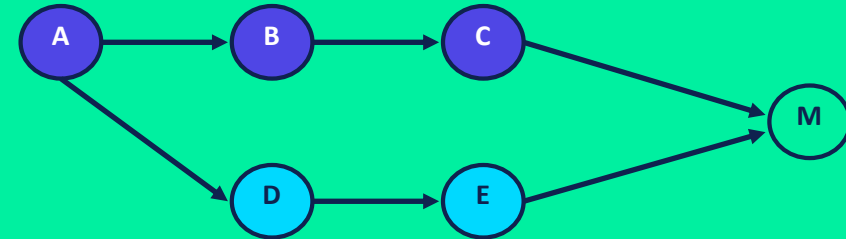
Merge into the branch you have checked out. It does not delete the source branch.

1. Fast-forward: no new merge commit



main moves from B → D; feature already points to D.

2. Three-way merge: two parents



GitHub PR merge choices

Merge commit = preserve the branch history.

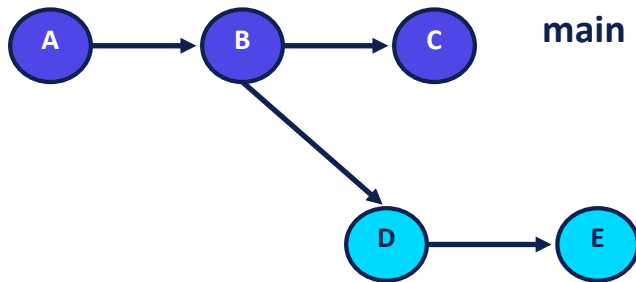
Squash = one combined change on main.

Rebase & merge = replay as a linear history.

Rebase: replay commits onto a new base

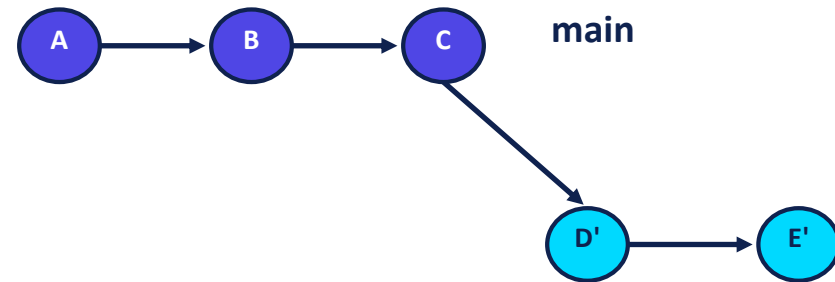
Rebase creates new commit IDs. Avoid rewriting commits other people already depend on.

Before: feature started at B



feature

After: feature rebased onto C



feature

```
git switch feature/prompt
git rebase main
git rebase --continue # after resolving
git rebase --abort    # cancel safely
```

MERGE: preserves branch history.

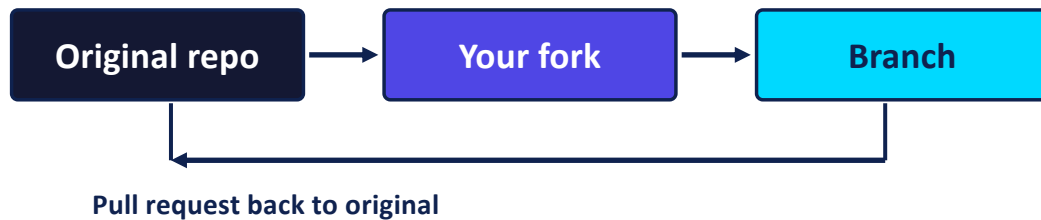
REBASE: rewrites selected commits.

Git workflows and branching strategies

Choose the simplest workflow that fits your team and release process.

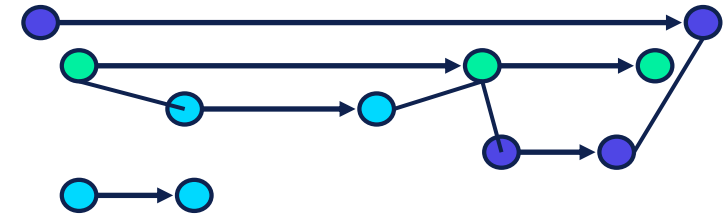
Workflow	Pattern	Useful when
Centralized	Everyone works against one main	Simple, carefully coordinated work
Feature branch	main → short branch → PR → main	Our beginner workshop
GitFlow	main + develop + release + hotfix	Scheduled/versioned releases
Forking	Own fork → branch → PR to original	Open-source / no upstream write access
Trunk-based	Tiny branches → frequent integration	Strong tests + fast feedback

Forking workflow



GitFlow: main, develop, feature, release, hotfix

main
develop
feature/*
release/*
hotfix/*





Conflicts: Git needs your decision

A conflict is normal when changes cannot be combined automatically.

Why it happens

Two changes touch the same content, or conflict through edits/deletions/renames.
Git cannot infer the correct final behaviour.

```
<<<<<< HEAD
Demo mode: cloud
=====
Demo mode: offline
>>>>>> demo/b
```

Resolve the intended final state

1. git status — find the conflicted paths.
2. Open VS Code Merge Editor or the file.
3. Edit, remove markers, save and test.
4. git add README.md
5. git commit (merge) / rebase --continue

```
git merge --abort
git rebase --abort
```

These cancel the current operation.





Undoing changes: restore, reset or revert?

First inspect status and history. Pick the tool that matches the state of your work.

Situation	Command	Result / risk
Staged the wrong file	<code>git restore --staged FILE</code>	Unstage; keep your edits
Discard unstaged tracked edits	<code>git restore FILE</code>	Edits are discarded; inspect first
Undo a local commit; keep staged	<code>git reset --soft HEAD~1</code>	Moves branch; keeps content staged
Undo a local commit; keep unstaged	<code>git reset --mixed HEAD~1</code>	Moves branch; keeps working edits
Reverse a shared commit	<code>git revert COMMIT</code>	New undo commit; shared history stays
Find a lost commit	<code>git reflog; git branch recovery COMMIT</code>	Preserve the found commit on a branch

DANGER: `reset --hard` and `clean -fd` can destroy local work. Preview cleanup with `git clean -nd`.





Advanced Git: stash, cherry-pick, debug and recover

These are optional power tools. Keep the safe everyday workflow as your default.

Command	Purpose	Important caveat
git stash push -u -m "WIP"	Park tracked + untracked unfinished work	Local; ignored files excluded
git stash list / apply / pop	Inspect / restore / restore then drop	Applying can conflict
git cherry-pick COMMIT	Reuse a selected commit's change	Creates a new commit; may conflict
git rebase -i HEAD~3	Edit/reorder/squash recent commits	Rewrites history; advanced use
git bisect start	Find the commit that introduced a bug	Mark known good/bad, then reset
git blame FILE / git reflog	Understand line history / ref history	Not recovery for unsaved edits

Recovery is safer when you make small commits and create a backup branch before rewriting history.



Git in real projects: habits and .gitignore

Clean commits, meaningful names and reviewable changes beat memorizing dozens of flags.

Good working habits

- Keep main runnable; use a branch per change.
- Make small commits with clear intent.
- Review diffs, test and resolve conflicts.
- Use tags/releases for known tested versions.

```
# .gitignore
.venv/
__pycache__/
.env
node_modules/
*.pem
```

Purpose	Branch / commit example
Feature / bug fix	feature/prompt-template fix/empty-input
Urgent / release / experiment	hotfix/timeout release/0.1 exp/model-b
Intent in a commit message	feat: add prompt template docs: explain setup

.gitignore does NOT erase tracked files or old secrets. Revoke exposed credentials immediately.



AI files: large artifacts, attributes and hooks

Version lightweight definitions; reference approved datasets and model artifacts.

Keep in Git / store elsewhere

Git: code, prompts, tests, small eval fixtures, configuration, docs and dependency versions.
External: private data, model weights, outputs.
Never commit: keys, tokens or credentials.

```
git lfs install
git lfs track "*.onnx"
git add .gitattributes
git lfs ls-files
```

```
# .gitattributes
* text=auto
*.png binary
*.jpg binary
```

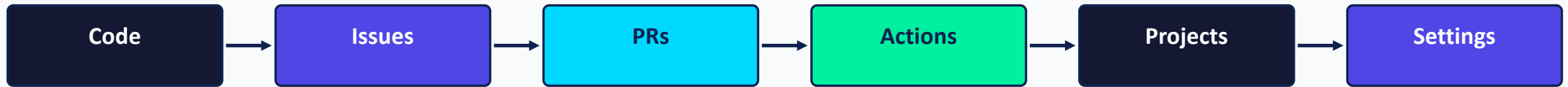
Hooks: automate local checks

Pre-commit / pre-push can run tests or linting.
Local hooks are not shared automatically.
Use CI too: local hooks can be bypassed.
README: document setup, tests and artifacts.

GitHub basics: repositories and the ecosystem

GitHub adds collaboration to Git. It is not the only cloud Git hosting platform.

OWNER / ai-git-lab ★ Star Fork Public / Private



GitHub features beginners should recognize

PRs = proposed code changes + reviews.
Issues = bugs, tasks and ideas.
Projects = work views; Actions = automation.
Releases, Discussions and Wikis add context.

Alternatives + why GitHub stands out

GitLab, Bitbucket Cloud, Azure Repos,
Codeberg and SourceHut also host Git.
GitHub: community + integrated collaboration
+ reusable automation and developer tooling.



GitHub Issues: bugs, defects, tasks and ideas

An issue records an outcome to investigate or deliver. A PR proposes changes to achieve it.

Type / label	What to capture
Bug / defect	Expected vs actual result + reproducible steps
Feature / enhancement	User need, proposed value, acceptance criteria
Task / maintenance	Specific deliverable, owner and definition of done
Docs / test / research	Missing guidance, coverage or question to investigate

Example: Handle empty prompt input

Problem: an empty prompt produces an invalid request.

Acceptance: reject blank input with a useful message.

Add: reproduction, context, assignee and priority.

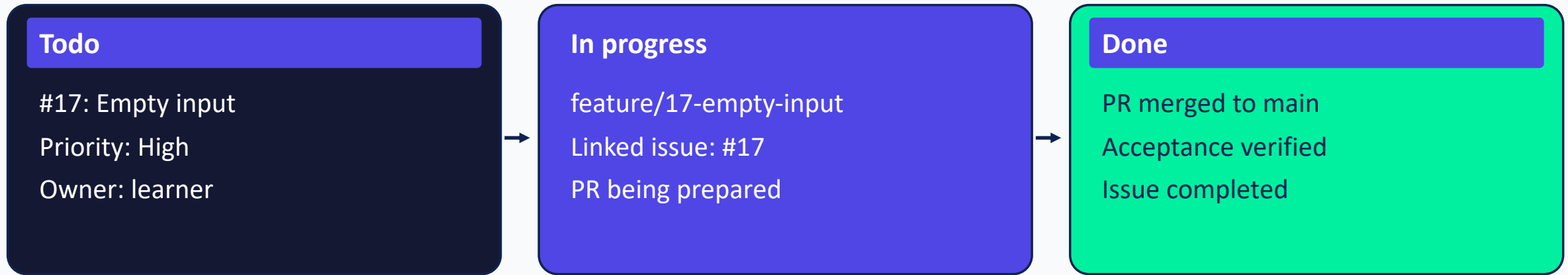
Keep confidential security reports private.

```
gh issue create --title "Handle empty input" --label bug
gh issue list --state open
gh issue view ISSUE_NUMBER --web
gh issue comment ISSUE_NUMBER --body "Reproduced locally"
```



Issues + Projects: work and its workflow

An issue is the work item. A Project is a board, table or roadmap spanning that work.



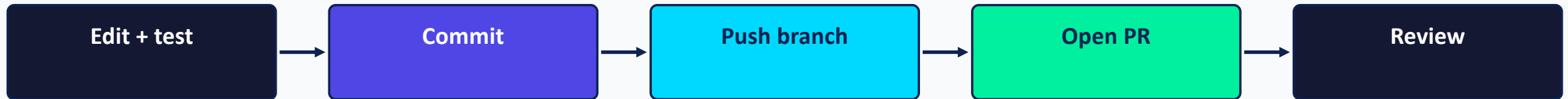
```
gh auth refresh -s project
gh project create --owner @me --title "AI App Lab"
gh project list --owner @me
gh project item-add PROJECT_NUMBER --owner @me --url ISSUE_URL
```

Link: issue → feature branch → PR body “Fixes #17” → merge to default branch → issue closes.



Open a pull request from the VS Code terminal

A PR compares a head branch with a base branch. Push your commits before asking for review.



```
git add prompts/system.txt
git commit -m "Clarify uncertain responses"
git push -u origin feature/ISSUE_NUMBER-prompt
gh pr create --base main
gh pr view --web
gh pr diff
gh pr checks
```

Write a useful PR body

WHAT: the behaviour changed.

WHY: the issue / user outcome.

TESTED: command + result.

LINK: Fixes #ISSUE_NUMBER

RISKS: limitations or follow-up work.



Review, merge or close a PR from the terminal

Merge integrates code. Close rejects or pauses it without merging. These are different outcomes.

Decision	Terminal command	Meaning
Inspect locally	<code>gh pr checkout PR_NUMBER</code>	Check out the proposal and run tests
Approve (reviewer)	<code>gh pr review PR_NUMBER --approve</code>	Reviewer approves; not self-approval
Merge approved work	<code>gh pr merge PR_NUMBER --squash --delete-branch</code>	Integrate and clean up the branch
Close WITHOUT merge	<code>gh pr close PR_NUMBER --comment "Superseded"</code>	Proposal is closed; main is unchanged
Reopen a proposal	<code>gh pr reopen PR_NUMBER</code>	Resume work where possible
Close completed issue	<code>gh issue close ISSUE_NUMBER --reason completed</code>	Mark the work completed

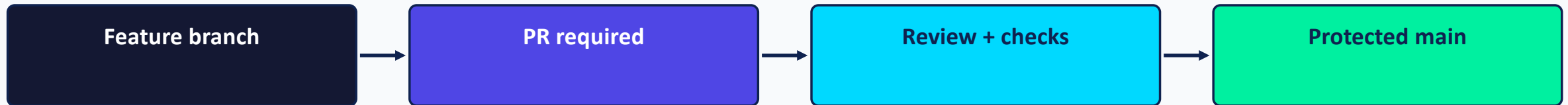
```
git switch main; git pull --ff-only
```

Before merging: review + tests + ruleset requirements.



Branch rulesets: protect main deliberately

Rules turn team expectations into enforced constraints; repository plan and permissions matter.



Configure a beginner-safe ruleset

Settings → Rules → Rulesets → new branch ruleset.
Target main; activate enforcement.
Require a PR; block force-push and deletion.
Require the exact passing CI check once it exists.

Do not accidentally block your workshop

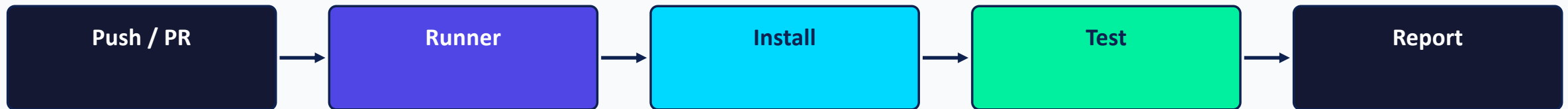
Approval requires another person, not the author.
Private repo features depend on the plan.
Keep bypass permissions minimal and explicit.
Check blocked merges; do not bypass blindly.

```
gh ruleset list
gh ruleset check main
```

Rulesets protect refs; branching strategy describes how the team works.

CI/CD briefly: where GitHub Actions fits

Keep the first pipeline small: verify a change before it reaches main. Deployment is a later session.



CI: prove the proposed change

Build/test each small integration.

Report pass or fail on the PR.

For AI apps: deterministic tests first;
add prompt/evaluation checks when appropriate.

CD: move a verified change toward users

Continuous delivery: ready to deploy; decision gated.

Continuous deployment: passing change auto-deploys.

This session demonstrates CI, not cloud deployment.

Actions also automates non-CI repository tasks.

Workflow → jobs → ordered steps | Trigger = event | Runner = execution machine



Write your first complete GitHub Actions workflow

Create `.github/workflows/ci.yml` in VS Code. Indentation matters: YAML expresses structure.

```
name: AI App CI
on: [push, pull_request]
permissions:
  contents: read
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: python -m pytest -q
```

Read the file from top to bottom

name: the label in Actions.

on: repository events.

permissions: minimal token access.

jobs: work run on a runner.

steps: checkout → Python → install → test.

A failing command fails the job.

```
git add .github/workflows/ci.yml
git commit -m "Add CI test"
git push
```



Variables, secrets and GitHub environments

Configuration is not always sensitive. Credentials must be handled differently.

Mechanism	Use / reference	Example
env	Shell value: \$NAME (Bash)	APP_MODE: test
vars	Non-sensitive settings: \${vars.NAME}	MODEL_NAME, REGION
secrets	Sensitive credential: \${secrets.NAME}	MODEL_API_KEY
environment	Job names a GitHub environment	staging / production + approvals

```
# In a job:
environment: staging
env:
  MODEL_NAME: ${vars.MODEL_NAME}
  MODEL_API_KEY: ${secrets.MODEL_API_KEY}
```

```
gh variable set MODEL_NAME --body demo
gh secret set MODEL_API_KEY
gh secret list
```

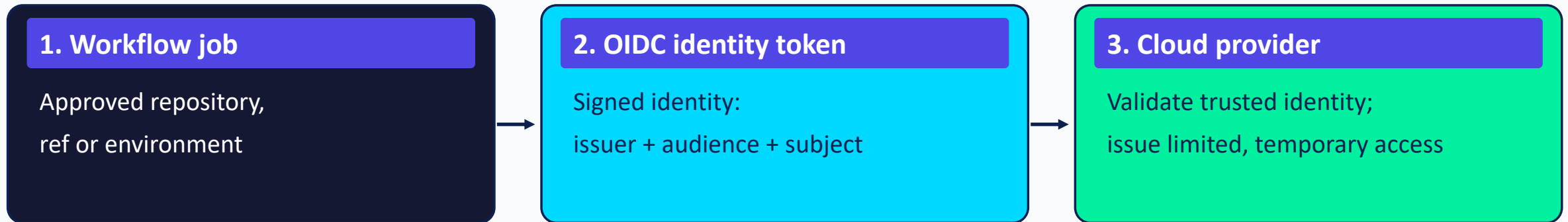
Enter secrets at the prompt.

Never print them or commit .env.



OIDC: cloud access without a stored long-lived key

OpenID Connect lets a workflow prove its identity and obtain short-lived cloud credentials.



```
# Job permissions (not a full login):
permissions:
  contents: read
  id-token: write
```

id-token: write allows token issuance; it does not grant cloud resource permissions.

Cloud trust must be configured too

- Trust a specific owner/repo + ref/environment.
- Restrict the cloud role to required resources.
- Use the cloud's supported login action.
- No static cloud key in the GitHub repository.

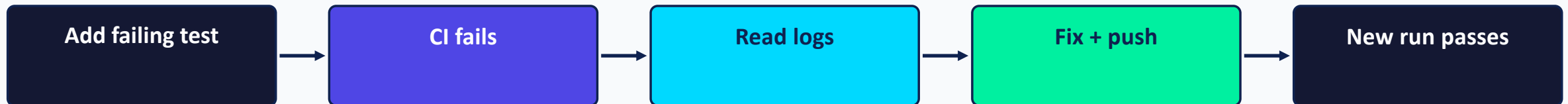
Troubleshoot a failed GitHub Actions pipeline

Use Lab 4: deliberate failing test → first failed command → fix + push → new run passes.

```
gh run list --limit 5
gh run view RUN_ID
gh run view RUN_ID --log-failed
gh run watch RUN_ID
gh run rerun RUN_ID --failed
```

First-error checklist

Correct branch + commit? Read the first failure.
YAML / paths / runtime / dependencies?
Tests reproduce locally with the same inputs?
Secret, token permission or approval missing?



Rerun uses the old run's commit. A code fix needs a new push and a new run, not just “rerun”.



Everyday Git commands: a practical lookup

Participant reference: [docs/command-reference.md](#). Full A–Z catalog is speaker-only.

Command	Purpose	Command	Purpose
add / bisect	Stage changes / locate a regression	blame / branch	Line history / branch references
checkout / cherry-pick	Older switch tool / replay a commit	clean / clone	Delete untracked (risk!) / copy repo
commit / config	Record snapshot / settings	diff / fetch	Compare / download remote refs
grep / help / init	Search / docs / create repo	log / merge	History / integrate branches
mv / pull / push	Move path / fetch+integrate / publish	rebase / reflog	Replay / local reference history
remote / reset	Manage connections / move/reset state	restore / revert	Restore files / new undo commit
rm / show / stash	Remove / inspect / park work	status / switch / tag	State / branch switching / version ref
worktree	Multiple working trees from one repo	git help -a	List all available commands





Continue learning: participant workbook

Every short exercise now has prerequisites, steps and success checks.

Labs 1–2: local Git

Clean history • branches • controlled conflict
Use a separate disposable practice folder.

Labs 3–4: GitHub + CI

Issue → branch → PR • controlled red-to-green CI
Use your own remote repository.

Lab 5: a tested release | Lab 6: Projects and governance (optional)

<https://github.com/imohweb/mmaug-bootcamp2026-learn-abcd-of-git-github>

Open the workbook → choose a lab → keep one traceable issue-to-tested-merge story.